M.Tech Program

Advanced Industry Integrated Programs

Jointly offered by University and LTIMindTree

Intelligent Cache Optimization for AI Agents

Review 1

DEV S SHAH
VIT CHENNAI
T9991

Knowledge partner

LTM

Implementation partner

L&T EduTech

Introduction

Problem

- Repeated LLM queries cause **high cost, latency, and token wastage**
- Scalability challenges due to growing API usage and rate limits

Solution: Intelligent Caching

- Exact Match Cache : Redis
- Semantic Similarity Cache : Pgvector
- Context-Aware Retrieval : Agent/Tool support.
- **Intelligent Policy Engine**: Cost/Frequency/Recency based eviction/caching.

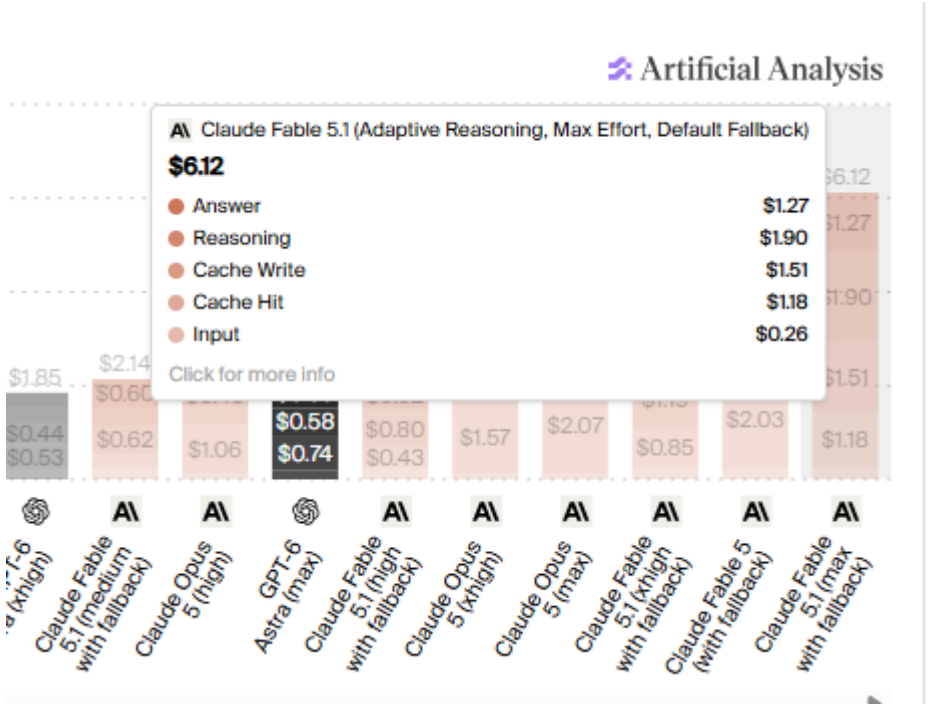
Objective

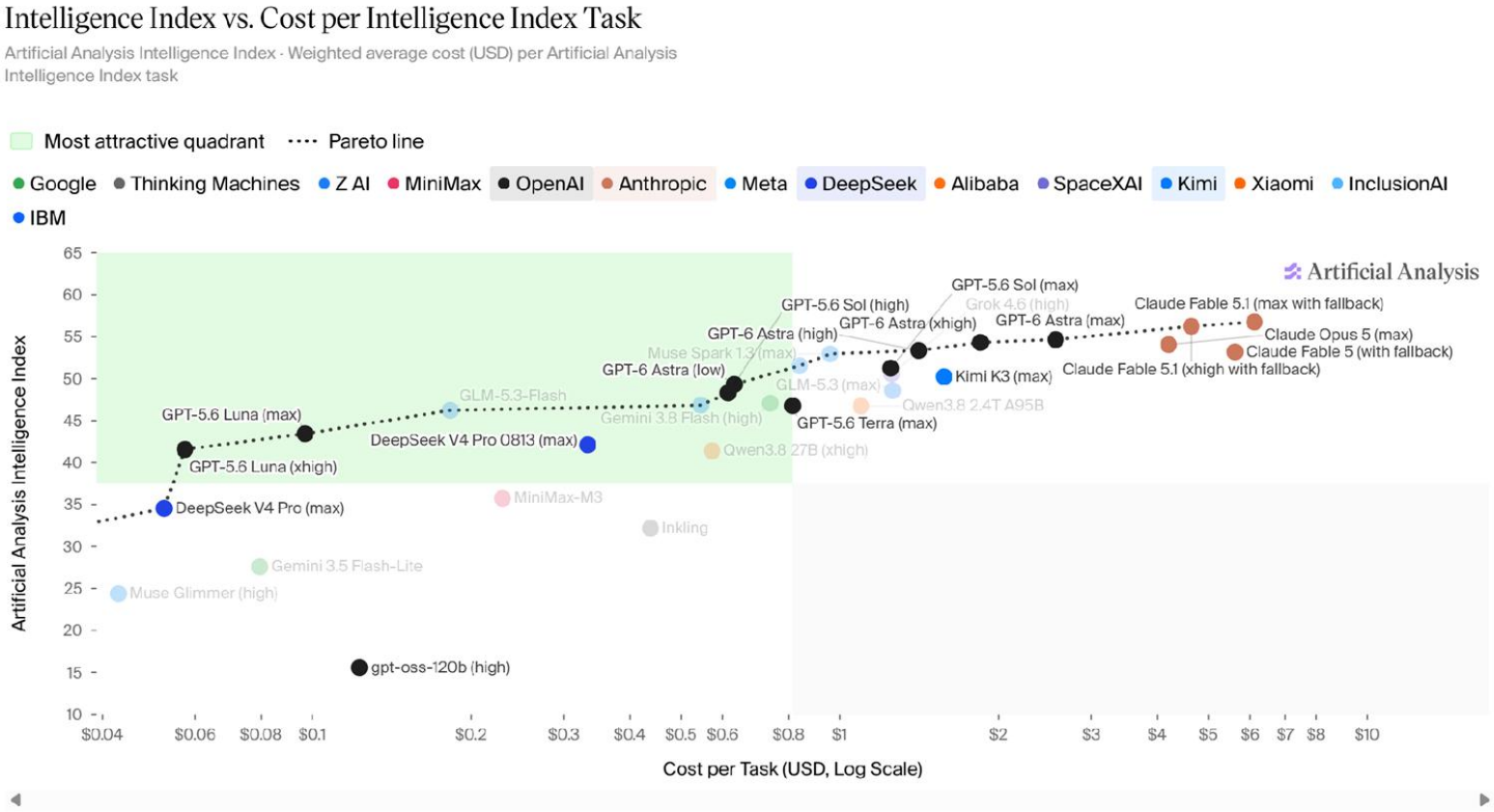
- **Tiered Cache Middleware**: Multi-stage interceptor for LLM/Agent calls.
- **Intelligent Storage Layer**:
  - *Exact:* Redis for deterministic lookups (low latency).
  - *Semantic:* Pgvector for context/meaning matches (prevents redundant compute).
- **Policy-Driven Optimization**: Intelligent scorer evaluates *if* result worth caching (balances cost vs. memory).
- **Observability Suite**: Streamlit dashboard tracking live hit-rates, latency, and cost savings.

Scope

- Support **exact-match and semantic similarity** based cache retrieval.
- Reduce **LLM inference calls**, latency, and API costs.
- Reuse deterministic outputs from tools and agent workflows.
- Implement context-aware cache invalidation and update policies.
- Measure performance using:
  - Cache Hit Rate
  - Response Time
  - Token Savings
  - Cost Reduction
  - Response Accuracy

Literature Review



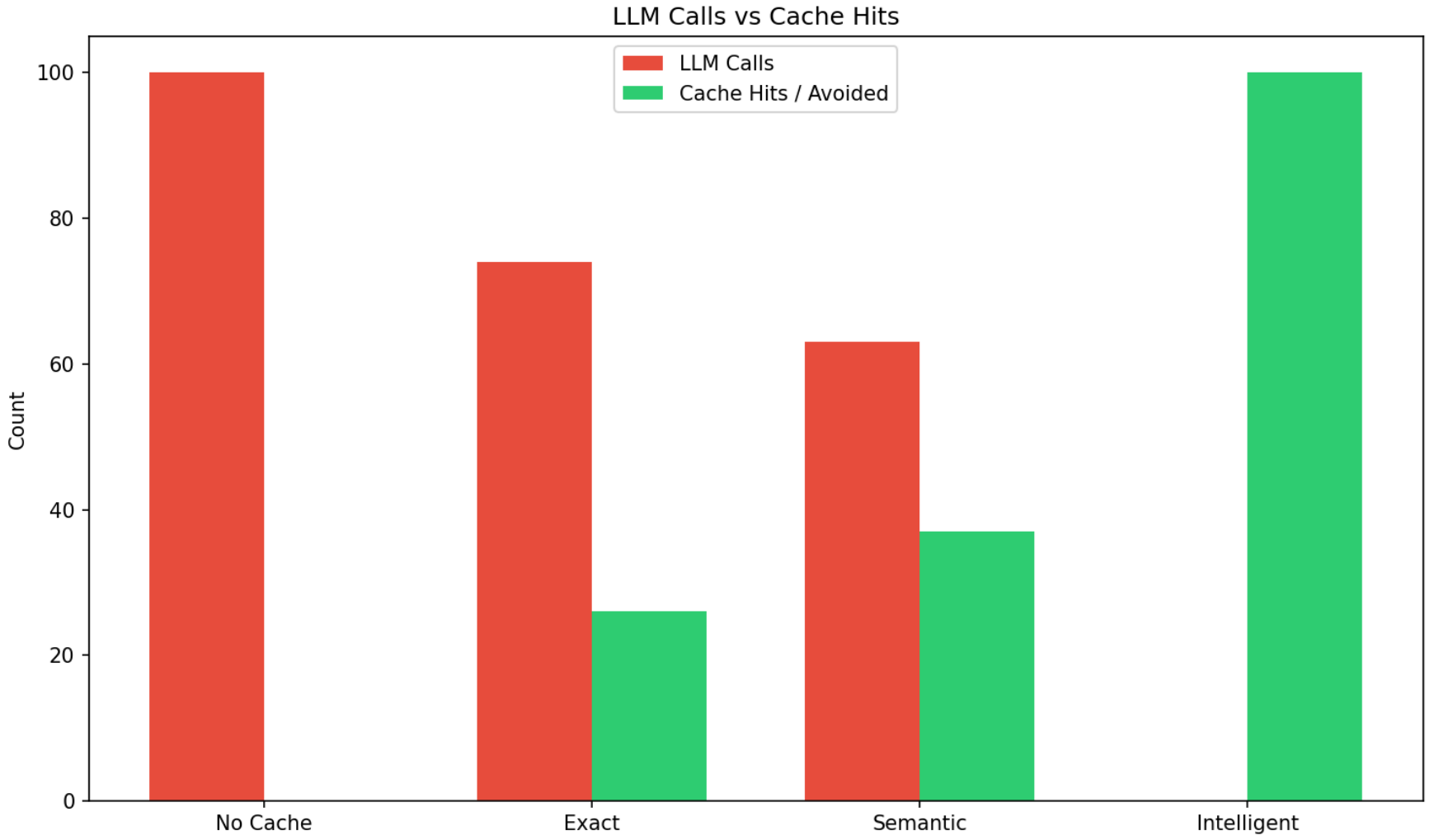


Summary of Literature

Key Findings from Literature

- Semantic caching significantly reduces repeated LLM calls and inference cost.
- Sentence embeddings (SBERT) enable accurate matching of paraphrased queries.
- Context-aware caching improves performance in multi-turn conversations.
- Existing approaches mainly focus on a single caching strategy (Exact or Semantic).
- Most systems lack explainable cache decisions and intelligent cache policies.

Additional Information



Modules to cover

- **Core Application Layer** → *API Management*
- **Cache Implementation Layer** → *Cache Operations*
- **Intelligence Layer** → *Decision Engine*
- **LLM Provider Layer** → *Model Connectors*
- **Supporting Components** → *Utility Services*
- **Dashboard & Monitoring** → *System Monitoring*
- **Benchmarking Framework** → *Performance Analysis*

Module name –Cache Manager

Dataset / Query Generation

- Generated a custom workload of user queries.
- Included exact, paraphrased, contextual, and repeated queries.
- Simulated real AI agent interaction patterns.

Cache Validation

- Tested Exact Cache for identical queries.
- Tested Semantic Cache for similar/paraphrased queries.
- Tested Context Cache for multi-turn conversations.
- Tested Tool Cache for deterministic tool outputs.